Portfolio Website: Development Process and File Guide

Project Overview

This is a personal portfolio website focused on architectural design, BIM, and building-environment research. It has two main pages: a portfolio page and a personal CV page. The site separates content from presentation: project and CV information is stored in JSON files, JavaScript loads the data and renders it, and HTML provides the page structure.

File Guide

File	Purpose
`index.html`	Portfolio page structure, navigation, category filters, project feed, footer, and project-detail modal.
`about.html`	About/CV page structure and footer. CV content is loaded dynamically by JavaScript.
`css/style.css`	Site-wide typography, colors, spacing, project layouts, interaction states, and responsive styles.
`data/projects.json`	Profile information, project categories, and project content such as year, summary, image, location, and details.
`data/cv.json`	CV information, including profile, education, work experience, skills, and honors.
`js/portfolio.js`	Loads project JSON, calculates category counts, renders project layouts, applies filters, and opens the project-detail modal.
`js/cv.js`	Loads CV JSON and renders the About/CV page. It also displays the email address as a clickable mail link.
`presentation.html`	A 16:9 web presentation about the project, with print styles for exporting to PDF.
`PRESENTATION_GUIDE.md`	Presentation outline and speaking prompts.
`README.md`	Project overview, data architecture, content-editing instructions, and deployment guidance.
`DEVELOPMENT_SUMMARY_ZH.md`	Traditional Chinese version of this development and file guide.
`DEVELOPMENT_SUMMARY_EN.md`	This English version of the development and file guide.

Development Process

1. **Define the purpose and visual direction**

Position the site as an architecture and computational-design portfolio. Use a clean, editorial style inspired by architectural publications, with one page for projects and another for the personal CV.

1. **Plan the data structure**

Store project and CV content in `data/projects.json` and `data/cv.json`. Use structured fields for categories, titles, years, images, summaries, and project details.

1. **Create the visual system**

Define typography, colors, spacing, project layouts, interactions, and responsive behavior in `css/style.css`.

1. **Build the rendering logic**

`js/portfolio.js` loads project data and creates category filters, featured projects, and grid projects. `js/cv.js` loads CV data and builds the CV page.

1. **Connect the pages and interactions**

`index.html` and `about.html` provide the page structure, while JavaScript inserts the content. The portfolio supports category filtering and a project-detail modal. The email address is provided as a `mailto:` link.

1. **Test and refine**

Check JSON syntax, image URLs, page paths, and interactive behavior. Since browsers generally prevent pages opened with `file://` from fetching JSON, run a local HTTP server for previews.

1. **Track revisions and prepare presentation materials**

Use Git to record development in stages, including specifications, styling, data, interactive features, profile updates, image fixes, contact details, and presentation revisions. Prepare `presentation.html` and the presentation guide for project reviews.

Main Features and Data Flow

```
data/projects.json ──> js/portfolio.js ──> index.html
                                       ├──> Category filters and counts
                                       └──> Project-detail modal

data/cv.json ──────────> js/cv.js ──────────> about.html
```

- Category filters calculate the number of projects from the project data.
- Featured and grid projects use different layouts, based on each block's `type`.
- Selecting a project opens a detail modal populated from its data.
- The contact email, `marc60809@gmail.com`, is available as an email link on the portfolio and CV pages.

Updating Website Content

Add or Edit a Project

1. Open `data/projects.json`.
2. Add or edit an entry in the `projects` array.
3. Provide a unique `id` and the required information, such as `title`, `year`, `category`, `summary`, and `image`.
4. Use `"type": "featured"` for a featured project. For a grid project, add it to the `items` array inside a `"type": "grid"` block.
5. Check the JSON quotes, commas, and braces, then refresh the portfolio page.

Update the CV

Open `data/cv.json` and edit the relevant fields in `profile`, `education`, `experience`, `skills`, or `honors`. Keep the existing data structure so `js/cv.js` can render the page correctly.

Change the Contact Email

If you change the email address, check the profile data in `data/projects.json` and `data/cv.json`, as well as the `mailto:` links in both page footers. This keeps the displayed address and recipient in sync.

Local Preview

Start a local web server from the project root, then open the server URL in a browser. For example:

```
python -m http.server 8000
```

Then visit:

- Portfolio: `http://localhost:8000/`
- CV page: `http://localhost:8000/about.html`
- Presentation: `http://localhost:8000/presentation.html`

If Python is unavailable, use the Live Server extension in VS Code. An `ERR_CONNECTION_REFUSED` message usually means the local server did not start successfully or the URL uses a different port; it does not necessarily indicate a problem with the website files.

Maintenance Notes

- Update content in the JSON files rather than hardcoding project or CV content in HTML.
- When adding a category, define it in the `categories` array in `data/projects.json` and use the same ID in each project's `category` field.
- Use valid, publicly accessible image URLs. The portfolio includes image error handling.
- JSON does not support comments. Check braces and commas before saving.
- Before presenting the project, update any placeholder deployment URLs or outdated sample information in the README and presentation guide.